

Setting up pmqtest: The pmqtest utility was written by Carsten Emde to measure the latency of interprocess communication with POSIX message queues. The program starts two threads that are synchronised using `mw_receive()`, `mw_send()`, and the time difference between receiving and sending the message is measured as latency.

The command used to start pmqtest is:

```
$ ./pmqtest -Sp 99 -i 100 -d 0
```

The 's' option implies 'a' and 't' options with the same priority on all threads. The explanation for the 'd' option is the same as given for ptsematest.

Setting up svsematest: The svsematest utility was written by Carsten Emde to measure the latency of SYS V semaphores. The program starts a pair of threads or, optionally, forks two processes that are synchronised via SYS V (System V) semaphores. The time difference between getting the semaphore on one side and releasing it on another side is measured as the latency.

The command used to start svsematest is:

```
$ ./svsematest -a -t -p99 -i100 -d0
```

Setting up cyclicttest: The cyclicttest utility was written by Thomas Gleixner. It measures the time that passes between when a timer expires and when the thread that set the timer actually runs. It does this by taking a time snapshot just prior to waiting for a specific time interval (t1), then taking another time snapshot after the timer finishes (t2), and then comparing the theoretical wakeup time with the actual wakeup time (t2 - (t1 + sleep_time)). This value is the latency for that timer wakeup.

The command used to start cyclicttest is:

```
$ ./cyclicttest -ss -p95 -m
```

The 's' option implies 'a' and 't' options with the same priority on all

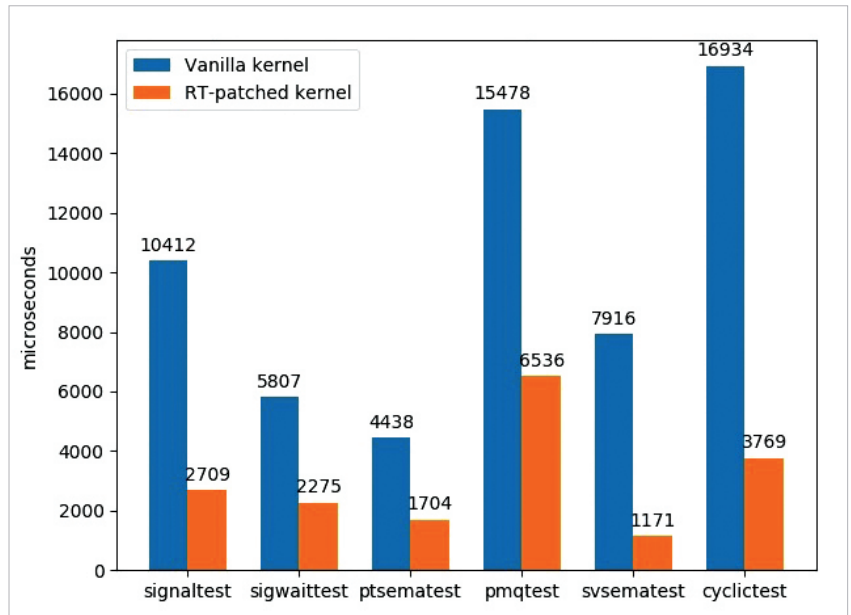


Figure 1: Comparison of worst case latency scenarios in kernels

threads. The 'm' option is to ensure that we lock the memory allocated to the process in the RAM so that it does not get paged out. Internally, the `mlock()` system call is used to implement this.

The results

We have run all the above mentioned tests and plotted a graph with the results obtained, which is shown in Figure 1.

The figure shows the PREEMPT_RT patch kernel performance is better than that of the vanilla kernel. The PREEMPT_RT patch provides preemptible critical sections. The preemptibility of normal spinlocks leads to blocking when trying to get a spinlock. This makes it illegal to get a spinlock with either disabled interrupts or preemption. In the PREEMPT_RT patch, the handlers of interrupts run in the process context and only the interrupts designated with SA_NODELAY are run in the interrupt context. It is to be noted that

only a few interrupts are designated with SA_NODELAY.

When working in real-time, priority inversion is always a challenge, because we don't want higher priority tasks waiting because a lower priority task has acquired a common shared resource and got preempted by a medium priority task. The popular solution to solve this problem is priority inheritance. In the PREEMPT_RT patch kernel, the lower priority task gets promoted to higher priority if it shares a common resource with the higher priority task. This allows high priority tasks to acquire the resource after releasing the shared resource to the lower priority task, which helps the former to avoid missing a deadline. The patch's priority inheritance is transitive, and also provides a way to handle situations where tasks with high priorities need resources or locks designated for low priority tasks. This is why the performance of PREEMPT_RT patch kernel is better than that of the vanilla kernel. **END** 🐧

By: Mohana Datta Yelugoti and B. Thangaraju

The authors are associated with the open source technology lab in the International Institute of Information Technology, Bengaluru.